

Ephemeral Memory

Successful Retention of Short-Term Information Using Variable Plasticity and Decaying Weight Values

Jaden Lorenc

Brigham Young University

Memorizing a Phone Number

1. hear it, fresh memory in its entirety
2. keep the important details
3. write it down
4. repeat it (aloud or in your head)
5. wait, forget parts of it
6. review

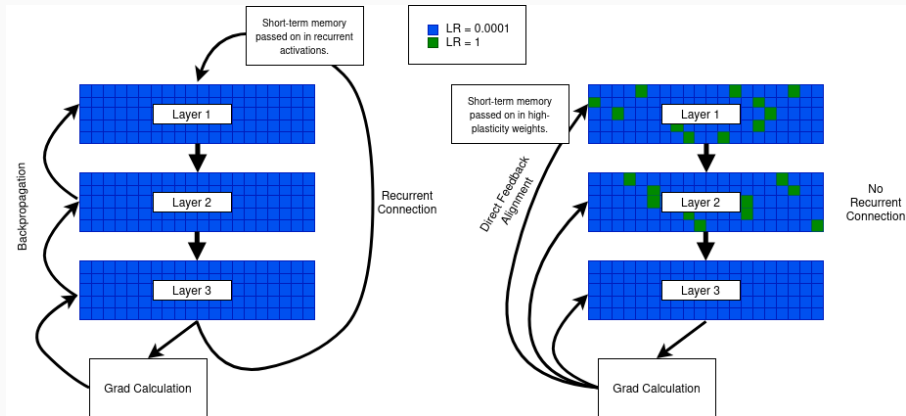
Existing Forms of “Memory”

human	NN Mechanism
hear it, fresh memory in its entirety	input activations, context window
keep the important details	attention mechanism, LSTM gate, small lr in pretraining
write it down	RAG, tool use
repeat it (aloud or in your head)	recurrent connection activation
wait, forget parts of it	weight decay, l2 norm
review	fine tuning, RL memory consolidation

Where It's Stored

human	NN Mechanism	Where It's Stored
hear it, fresh memory in its entirety	input activations, context window	input activations
keep the important details	attention mechanism, LSTM gate, small lr in pretraining	attention matrix (activations), recurrent activations
write it down	RAG, tool use	database
repeat it (aloud or in your head)	recurrent connection activation	activations
wait, forget parts of it review	weight decay, l2 norm fine tuning, RL memory consolidation	weights weights!

Can Short-term Information be Stored in the Weights?



Experiments

Tasks:

- Repeated Sequences
- Key-Recall
- Reversed Sequences

Key-Recall examples:

0000?10!1

00000?200!2

00?,00! ,

00?.0! .

Reversed examples:

abcba

xyzyx

12321

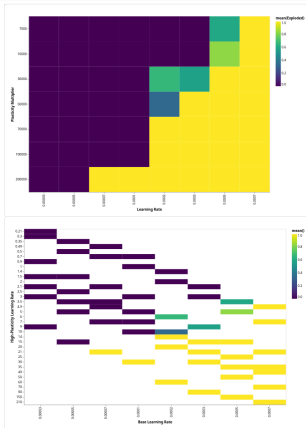
qwertytrewq

Inherent Volatility

- Both **exploding gradients** and **ethereal-weight volatility** arise from **positive feedback** in the update rule.
- As **parameter magnitudes increase**, the **induced gradients** also grow, amplifying subsequent updates.
- Without **explicit damping** (e.g., weight decay, normalization, clipping), the **effective gain** exceeds 1, leading to runaway growth.

Stabilizing

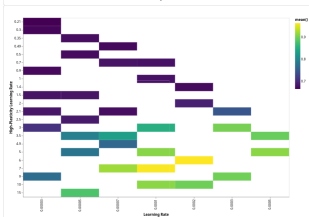
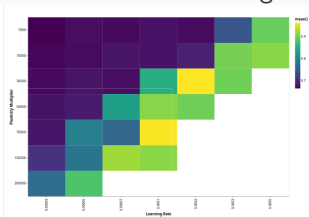
With weight decay and careful hyperparameter tuning, this can be stable!



Accuracy

While aggressive forgetting rates prevent explosion, they also reduce memory duration below useful thresholds.

Conversely, insufficient forgetting leads to inevitable instability. This creates a narrow operational window that varies significantly across tasks and datasets.



Multi-Timescale Learning

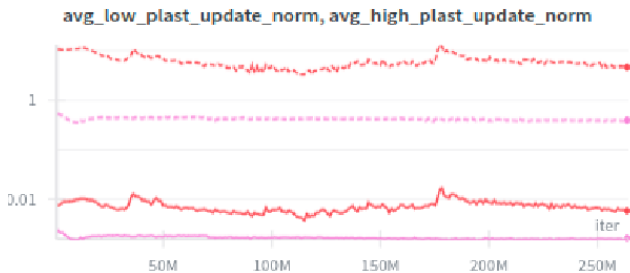


Figure 7: Gradient norm evolution throughout training shows clear separation between high-plasticity and low-plasticity weight categories. The high-plasticity weights (red) exhibit characteristic volatility with occasional spikes corresponding to memory encoding events, while low-plasticity weights (blue) maintain steady, controlled gradient magnitudes. This separation demonstrates that the two learning processes operate independently, allowing stable long-term learning to proceed alongside volatile working memory encoding.

The Control Problem

We want to control the plasticity in those fast weights:

$$x := \|w_{\text{eph}}\|.$$

Keep it in the corridor, so we can scale, fiddle with lr , whatever.

This is just a single scalar! Very basic info. (not a hessian)

- $\delta x = x - \bar{x}$ — state deviation
- $\delta u = \alpha - \alpha_0$ — control deviation (what the controller picks)
- $\delta d = g_{\text{raw}} - \bar{g}$ — disturbance deviation (what we can't control)

$$\delta x(t+1) = A \delta x(t) + B \delta u(t) + E \delta d(t).$$

A is $\partial(\text{update})/\partial x$ at the operating point: how a kick to the weight norm propagates one step forward. B is $\partial(\text{update})/\partial \alpha$: sensitivity of the next-step weight norm to the plasticity knob. E is $\partial(\text{update})/\partial g_{\text{raw}}$: sensitivity of the next-step weight norm to a fluctuation in gradient norm.

This all causes problems later bc A is time-varying.

Attempts to Stabilize

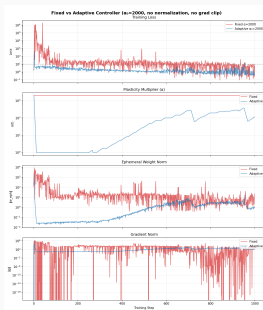
Treat α as a control input and $\|w_{\text{eph}}\|$ as the state. Three controllers, all reading the live weight norm and choosing α at each character step:

Controller	Idea	Channel used
LQR	Solve discrete Riccati once, apply $u = -K \delta x$	A, B — ignores E by construction
H_∞	Minimize worst-case disturbance gain	A, B, E — full partitioned plant
Adaptive	Heuristic gain scheduling, updates K online from observed error	doesn't trust the linearization

LQR is the textbook starting point. H_∞ adds robustness against the gradient-norm disturbance. Adaptive is the one that doesn't take the LTI model too seriously and just reacts to what it sees.

Results — Palindrome Reversal

- **Fixed** α (red): blows up above the corridor; fails to memorize below it
- **LQR**: stable in a narrow band around the operating point
- H_∞ : slightly wider stable band, conservative response
- **Adaptive** (green): stable across the widest α range; catches incipient runaways before the linearization fails



What Didn't Work

1. **Linearization doesn't last.** A is fitted at one operating point. Once the trajectory leaves that neighborhood, the model is wrong and the controller is solving the wrong problem.
2. **A is time-varying.** The loss landscape morphs as training progresses; we identify A once and freeze it.
3. $\|w_{\text{eph}}\|$ **isn't the state we actually care about.** Accuracy is not monotone in weight norm — you can keep the corridor and still lose the memory.
4. H_{∞} **is conservative.** The worst-case budget gets spent on disturbances we couldn't actually afford to hedge against.
5. **Diagnosis:** gray-box LTI on a scalar state is the wrong abstraction for a system whose load-bearing dynamics live in the *time-varying* part.

Why It Matters Anyway

- **Demonstrates** that classical robust-control machinery applies to learning dynamics with minimal assumptions.
- The **gray-box recipe** (analytical structure + one-parameter fit) is reusable for any plasticity-modulated training rule.
- The **failures are diagnostic**: they tell us exactly what's needed next.
- **First step** in connecting robust control theory to plasticity scheduling in neural networks — a research direction that's wide open.

- **μ -synthesis** to handle time-varying A as structured uncertainty
- **MPC** to express the corridor as a *hard constraint*, not a soft cost
- **Online identification** to track A as the loss landscape morphs
- **Vector-valued state** to capture the directional information lost when collapsing to $\|w_{\text{eph}}\|$
- **Connection to neuroscience**: short-term synaptic plasticity (STSP) has the same runaway-vs-decay tradeoff and may motivate the right state representation

- Ephemeral weights store short-term info, but the corridor between *fail-to-memorize* and *runaway* is narrow
- A linearized scalar plant captures the runaway dynamics with a single load-bearing parameter (the effective Hessian H)
- Adaptive gain scheduling beats fixed α ; LQR and H_∞ work in narrower regimes
- Right next step: μ -synthesis with online identification

Thanks — questions?